

Roadmap do Projeto e Base Cientifica

Sistema web em Python/Flask para extrair conhecimento probabilistico de uma base categorica, montar restricoes de programacao linear e responder perguntas condicionais feitas pelo usuario.

1. Objetivo geral

O objetivo implementado foi construir uma aplicacao dinamica em que o usuario escolhe afirmacoes e perguntas na interface. O backend calcula probabilidades, suporte, confianca, lift, intervalos e resolve a consulta por programacao linear usando o dataset de recomendacao de culturas.

2. Roadmap do que foi feito

Etapas	Implementacao realizada	Evidencia no projeto
Dataset	Foi mantido apenas o dataset do solo/culturas no repository do github	data/crop_recommendation.csv
Interface	Tela para escolher evento A e condicoes B dinamicamente	index.html, styles.css, script.js
Backend Python	API Flask para metadados, consulta e health check.	app.py
Deteccao dinamica	O CSV e lido; colunas numericas e categoricas sao detectadas automaticamente.	load_data.py
Discretizacao	Atributos numericos viram categorias baixo, medio e alto automaticamente.	category_mapper.py
Probabilidades	Calculo de P(A), P(B), P(A e B) e contagens.	probability(), probability_count()
Regras	Suporte, confianca/precisao e lift sao calculados para a regra	api/query
Intervalos	Probabilidades arredondadas em tres casas viram limites de intervalo	round_interval()
Programacao linear	Cada mundo possivel vira x_w; marginais e consulta sobre restricoes	solve_restriction()
Solver	SciPy linprog com metodo HiGHS resolve minimo e maximo	scipy.optimize.linprog
Saida	Interface mostra metricas, programa linear e tempo de processamento	api/query
Relatorios	PDF com graficos, explicacao e JSON dos calculos matematicos.	reports
Deploy	Configuracao Render Web Service Python com Python 3.11.9 final	render/python-version

3. Fluxo de processamento

O fluxo operacional do sistema e:

1. Usuario escolhe A e B na interface.
2. Frontend envia JSON para /api/query.
3. Python valida os valores contra os dominios reais do CSV.
4. Python calcula P(A), P(B), P(A e B), suporte, confianca e lift.
5. Python monta restricoes lineares com intervalos de tres casas decimais.
6. O solver calcula minimo e maximo de P(A | B).
7. A interface apresenta metricas, intervalo, formulacao linear e tempo.

4. Algoritmos matematicos usados

Algoritmo	Formula	Uso no projeto
Discretizacao por quantis	$x \leq Q1$: baixo; $x \leq Q2$: medio; $x > Q2$: alto	Transforma base numerica em base categorica.
Marginal	$P(E) = \text{count}(E) / N$	Calcula frequencia de qualquer valor de atributo.
Conjunta	$P(A \text{ e } B) = \text{count}(A \text{ e } B) / N$	Calcula suporte e restricoes da regra.
Condicional	$P(A B) = P(A \text{ e } B) / P(B)$	Representa a pergunta do usuario.

Algoritmo	Formula	Uso no projeto
Suporte	$\text{support}(B \rightarrow A) = P(A \text{ e } B)$	Mostra quanto a regra aparece na base.
Confianca	$\text{confidence}(B \rightarrow A) = P(A B)$	Mostra precisao da regra.
Lift	$\text{lift} = P(A B) / P(A)$	Compara a regra com a chance geral de A.
Intervalo	$\text{round}(p,3) - 0,001 \leq P(E) \leq \text{round}(p,3) + 0,001$	Evita rigidez numerica e representa incerteza intervalar.
LP	$\min c^T x \text{ sujeito a } A_{\text{ub}} x \leq b_{\text{ub}}, A_{\text{eq}} x = b_{\text{eq}}$	Formato usado pelo scipy.optimize.linprog.
Charnes-Cooper	$P(A \text{ e } B)/P(B) \rightarrow x_w = y_w/t, P(B) \text{ em } y = 1$	Transforma razao condicional em objetivo linear.

5. Base cientifica e tecnica dos links

Fonte indicada	Ideia aproveitada	Aplicacao no projeto
Mathematical programming models	Problemas de otimizacao podem ser formulados como problemas de programacao linear	Probabilidades das regras da base transformadas como restricoes de programacao linear
ScienceDirect: Artificial Intelligence, Base 0006678294906795	Base de conhecimento para representar conhecimento	Justificacao para limites inferiores/superiores, nao apenas um ponto
B. Tessem, Interval probability propagation (1992) 185-120	Probabilidades (1992) 185-120 apresentam intervalos em tres casas para P(A), P(B) e P(A B)	Cesta para limites
Laplace smoothing	Evita probabilidade zero em estimativas por frequencia	Fórmula implementada como extensao cientifica para bases com zeros
Material sobre verossimilhanca	Probabilidades podem ser estimadas por frequencia	$P(E) = \text{count}(E) / \text{count}(\text{tudo})$ estimador empirico das probabilidades
SciPy linprog	Define LP como minimizacao linear com restricoes	Se resolve o problema de programacao linear com o scipy.optimize.linprog(metodo)
Gurobi - Linear Programming in Python	Fluxo geral: definir variaveis, funcao objetivo, restricoes	O projeto resolve o problema de programacao linear com o SciPy como solver aberto
Ip_solve	Solver MILP/LP livre baseado em simplex	Enviado para referencia e teste de desempenho futuro de outros solvers
COPT-Public/cuPDLP-C	Referencia moderna para resolvidores de LP	Para benchmark de tempo, tamanho e desempenho

5.1 Como a fundamentacao virou codigo

A implementacao foi organizada como uma cadeia de inferencia probabilistica. Primeiro, o dataset é carregado e categorizado. Depois, o sistema minera frequencias empiricas para calcular probabilidades marginais, conjuntas e condicionais. Em seguida, a consulta escolhida pelo usuario é interpretada como uma regra explicita $B \rightarrow A$, com suporte, confianca e lift.

Na etapa de programacao linear, cada mundo possivel w recebe uma variavel x_w . A normalizacao $\text{soma}(x_w) = 1$ garante uma distribuicao valida. As probabilidades mineradas são transformadas em restricoes intervalares do tipo $L \leq \text{soma}(x_w \text{ onde o evento ocorre}) \leq U$. Isso inclui marginais, conjuntas por pares, restricoes da consulta seleccionada e, quando houver suporte positivo, a regra de associacao seleccionada.

A consulta $P(A | B)$ é uma funcao fracionaria, pois depende de $P(A \text{ e } B)/P(B)$. Para manter o problema compativel com um solver de programacao linear, o projeto usa a transformacao de Charnes-Cooper. Depois da transformacao, o HiGHS, chamado por `scipy.optimize.linprog`, resolve duas otimizacoes: uma para o limite inferior e outra para o limite superior da probabilidade condicional.

A validacao foi implementada com um solver separado. A interface envia a mesma consulta para esse script independente e compara suporte, confianca, lift, intervalo linear e tempo de execucao. Desse modo, o projeto mostra tanto o resultado principal quanto uma verificacao externa da montagem do modelo.

6. Justificativa das escolhas

A escolha de SciPy/HiGHS foi feita porque o trabalho precisava de um solver executavel em Python e adequado ao Render. A documentacao do SciPy define exatamente o formato usado: objetivo linear, restricoes $A_{\text{ub}} x \leq b_{\text{ub}}$, restricoes $A_{\text{eq}} x = b_{\text{eq}}$ e limites para variaveis. Como $P(A | B)$ é uma razao, a consulta foi transformada por Charnes-Cooper para manter o problema linear.

7. Sairas geradas

Arquivo	Conteudo
reports/relatorio_saida_programacao_linear.pdf	Explicacao, graficos, algoritmos matematicos, metricas e LP.
reports/saida_calculada_programacao_linear.json	Resultados numericos, contagens, tempos e calculos por algoritmo.
reports/roadmap_base_cientifica.pdf	Roadmap completo e base cientifica/técnica do projeto.
/api/query	Saida dinamica com suporte, confianca, lift, intervalo linear e tempo de processamento.

8. Proximos passos sugeridos

Para evoluir o projeto, os proximos passos sao: adicionar Laplace smoothing como opcao na interface; permitir conjuntas de pares/trios como nivel de restricao; comparar SciPy, Gurobi, lp_solve e outros solvers; medir curva de crescimento de tempo por numero de variaveis/restricoes; e calcular acuracia quando o atributo escolhido for uma classe.

9. Links de referencia

ResearchGate: https://www.researchgate.net/publication/260908286_Mathematical_programming_models_for_reasoning_under_uncertainty
ScienceDirect: <https://www.sciencedirect.com/science/article/abs/pii/S0004370294900795>
Tessem IJAR: Interval probability propagation, International Journal of Approximate Reasoning 7 (1992) 95-120
ScienceDirect: <https://www.sciencedirect.com/science/article/pii/S0165489625000241>
SciPy linprog: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html>
Gurobi LP Python: <https://www.gurobi.com/resources/blog/introduction-to-linear-programming-in-python>
Gurobi docs: <https://docs.gurobi.com/current/>
lp_solve: <https://sourceforge.net/projects/lpsolve/>
cuPDLP-C: <https://github.com/COPT-Public/cuPDLP-C>
Laplace smoothing: materiais indicados no Classroom e referencias complementares.